CS3081 Cloud Computing: Mini Project Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Item	Details
Student	Niraj Kumar
Roll number	230101037
Course	CS3081 Cloud Computing
Institution	Indian Institute of Information Technology Senapati, Manipur
Repository	https://github.com/NirajS2118/cloud-assignment

1. Introduction and Project Overview

The goal of this project was to take a large language model that is expensive to run and teach a much smaller model to do one narrow job well. The job is command parsing: a short voice-style command goes in, and a JSON object with four fields comes out (`intent`, `date`, `time` and `place`). There are eight possible intents. The big model is the teacher (Phi-3.5-mini-instruct, 3.8B parameters) and the small model is the student (EleutherAI/Pythia-70M, about 70M parameters). The student is then packaged as a web service and run on a Kubernetes cluster on my own machine.

The work is split into seven tasks. Tasks 1 and 2 are about data and training: storing the commands, labelling them with the teacher, cleaning the labels and training the student with sequence-level knowledge distillation. Tasks 3 and 4 turn the student into a FastAPI service, put it in a Docker image and deploy it on a k3d Kubernetes cluster. Task 5 is a paper-and-simulator exercise on subnetting and network ACLs. Task 6 covers the Makefile, tests and GitHub Actions pipeline, and Task 7 collects the performance numbers, the cost estimate and a review. Figure 1 shows how the pieces connect.

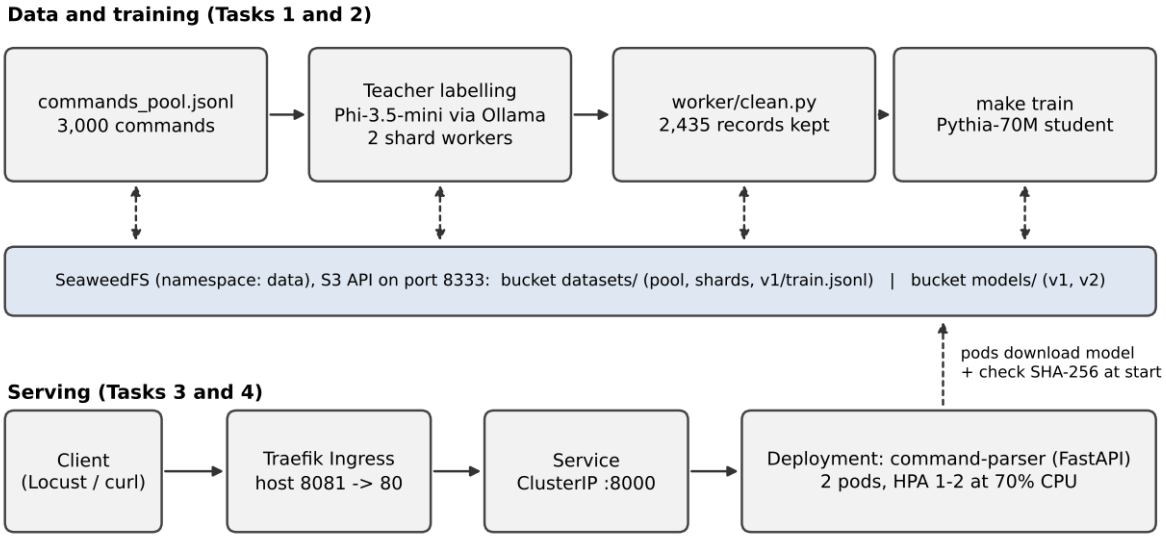


Figure 1. Overall architecture of the project.

Throughout the report I try to keep three things separate: **measured** results (numbers I recorded from runs), **estimated** values (the cost figures, which come from a rate card and a formula) and **proposed** improvements (ideas I did not implement or test). All numbers are the ones from my project evidence. Where two figures in my own material do not agree, I say so in the text.

2. Task 1: Object Storage and Parallel Teacher Labelling

This task needed a place to keep the data and a way to label 3,000 unlabelled commands with the teacher without a single worker doing everything. I started with the cluster. Using `cluster/k3d.yaml` I created a k3d cluster with one server node (`k3d-cloud-assignment-server-0`) and one agent node (`k3d-cloud-assignment-agent-0`). The built-in load balancer maps host port 8081 to port 80 and host port 8444 to 443.

For storage I deployed SeaweedFS in the `data` namespace (`k8s/storage/seaweedfs.yaml`). It gives an S3-compatible API on port 8333 and uses a 5 GiB PersistentVolumeClaim. It suits the project because the code can talk to it with boto3, the same way it would talk to a public cloud bucket. I created two buckets: `datasets` for the command pool and teacher output, and `models` for student weights and metadata. The 3,000 commands (`data/commands_pool.jsonl`) went to `s3://datasets/commands_pool.jsonl`.

The teacher runs locally through Ollama with `temperature: 0` and `seed: 42`, so that labels are repeatable. The prompt in `prompts/teacher_prompt.txt` asks for exactly one JSON object with the four keys, using one of the eight intent classes, and no extra text. To avoid a single-worker bottleneck I ran two jobs, `teacher-worker-0` and `teacher-worker-1`, in the `pipeline` namespace. Each one reads `SHARD_INDEX` and `SHARD_COUNT=2`, downloads the pool, and keeps only the lines where `line_number % SHARD_COUNT == SHARD_INDEX`. Worker 0 labelled 1,500 even-indexed commands into `datasets/teacher/shard-0.jsonl` and worker 1 labelled 1,500 odd-indexed ones into `shard-1.jsonl`. I checked the output afterwards: no command appears in both shards, and all IDs from C0001 to C3000 are covered.

The teacher is not perfect, so the labels need cleaning before they can be used for training. `worker/clean.py` goes through the shards in ID order and applies five rejection rules. Table 1 shows where all 3,000 records ended up, and the numbers add up exactly: 75 + 4 + 82 + 359 + 45 + 2,435 = 3,000.

Table 1. Cleaning accounting for the 3,000 raw teacher records (measured).

Category	Rule / action	Records	Share
Input	Raw records from shard-0 and shard-1	3,000	100.00%
Reason (a)	Reply is not valid JSON	75	2.50%
Reason (b)	Keys are not exactly intent, date, time, place	4	0.13%
Reason (c)	Intent is not one of the eight valid intents	82	2.73%
Reason (d)	Slot value is neither null nor an exact substring of the command	359	11.97%
Reason (e)	Duplicate command (lowest ID kept)	45	1.50%
Kept	Written to `datasets/v1/train.jsonl`	2,435	81.17%

The biggest loss was reason (d), almost 12% of the data. The teacher often rephrased a date or place instead of copying the words from the command, and the specification requires slot values to be exact substrings of the command, so those records were rejected. The SHA-256 of the training file is stored in `datasets/v1/train.sha256`:

```
ee52ae7ba8a69bf2903dc828f69c46e5271a5c28715a041b32964668d79a5daf
```

One thing I could improve here is to force the teacher to produce valid JSON at decoding time (for example with a grammar-constrained decoder) so that fewer records are thrown away afterwards.

3. Task 2: Training the Student with Sequence-Level Knowledge Distillation

The task was to train the 70M student on the teacher's labels and to explain why this style of distillation was used. In sequence-level distillation (Kim and Rush, 2016) the student is simply trained with ordinary next-token cross-entropy on the text the teacher produced. If x is a command and y is the teacher's JSON output, the loss is:

$$L_{seq} = -\sum_{t=1}^{T} \log P_S(y_t \mid y_{<t}, x)$$

In logit-based distillation the student instead tries to match the teacher's full probability distribution at every token, using a KL-divergence term with a temperature τ:

$$L_{logit} = (1 - \alpha)\, L_{CE} + \alpha\, \tau^2\, \mathrm{KL}(\, \mathrm{softmax}(z_T/\tau) \,\|\, \mathrm{softmax}(z_S/\tau)\,)$$

I could not use the logit-based version in this project, for two reasons. First, the two models do not share a vocabulary: Phi-3.5-mini has 32,064 tokens and Pythia-70M has 50,304, with different token boundaries, so the teacher's logits cannot be lined up with the student's without an arbitrary mapping. Second, the teacher is served through Ollama's `/api/generate` endpoint, which returns decoded text only and not the logits. Sequence-level distillation needs nothing but text, so it fits both limits.

It does have a weakness, which I think matters for a model this small. The student only sees the teacher's single chosen output and never learns how unsure the teacher was. It is also trained with the correct previous tokens

but must use its own previous tokens at inference time, so one early mistake (a missing brace, for example) can push the rest of the output off track. This is the exposure-bias problem.

The six required training settings are in `training/config.yaml`: model EleutherAI/pythia-70m, seed 42, AdamW with learning rate 5×10^{-5}, batch size 4, one epoch, and maximum sequence length 128 tokens. Library versions are pinned to Python 3.11.9, PyTorch 2.5.1, Transformers 4.46.3 and Boto3 1.35.71. `make train` runs the full training on the cleaned data, and `make train SUBSET=50` runs a quick check on 50 records in under 15 seconds, which is a quick way to check the training code. The average loss for the single epoch was 0.6519 (`training/loss.csv`). Weights and tokenizer were saved to `s3://models/v1/` and `s3://models/v2/`, and the checksum of the v1 weights is in `models/v1/model.sha256` (`50f44f9aeb3e8ad0d208ede36a2ae1cf402b3b88095f0d9f822f3926325ff80a`).

I scored the teacher, the untrained Pythia-70M and the trained student on the 100 gold commands in `data/validation.jsonl` with `tools/score.py`.

Table 2. Evaluation on 100 validation commands (measured).

Metric	Teacher	Untrained student	Trained student
JSON validity	94%	0%	99%
Intent accuracy	75%	0%	53%
Date accuracy	74%	0%	51%
Time accuracy	80%	0%	80%
Place accuracy	79%	0%	84%
Full-frame accuracy	45%	0%	16%

The untrained Pythia-70M scored 0% everywhere because it just produced repeated, unstructured text. After distillation the student gives valid JSON 99% of the time, which is above the teacher's 94% and above the 95% quality gate used in CI. It matches the teacher on time (80%) and is higher on place (84% against 79%), though with only 100 test commands a gap of five points is small and I would not read much into it. Where the student is clearly weaker is intent (53%), date (51%) and full-frame accuracy (16%), where all four fields must be right at once. The full-frame score is low for the teacher too (45%), so the task is harder than the field-level numbers suggest. My reading is that 70M parameters are enough to learn the output format and copy simple spans, but not enough for date reasoning and telling similar intents apart. The 70M student is far from a replacement for the teacher in accuracy; it is only much cheaper to run.

4. Task 3: Building and Containerizing the Inference Service

This task needed a small HTTP service around the student that could be built into a Docker image under 2.0 GB, with the model kept outside the image. I wrote it with FastAPI in `service/app.py`, listening on port 8000, with three endpoints:

- `POST /parse` takes `{"text": "..."}`, generates with Pythia-70M on the CPU, parses the JSON, checks it against `service/schema.json` and returns the four-key object. It adds the headers `X-Model-Version` and `X-Served-By` (the pod hostname).
- `GET /healthz` returns 200 as long as the process is up. This is the liveness probe.
- `GET /readyz` returns 503 while the model is still downloading and 200 only after the weights are loaded and verified. This is the readiness probe.

Configuration comes from four environment variables: `MODEL_URI` (for example `s3://models/v1/`), `MODEL_VERSION`, `S3_ENDPOINT` (`http://seaweedfs.data.svc.cluster.local:8333`) and `MAX_NEW_TOKENS` (100). The S3 credentials are not in the code or the image; they come from a Kubernetes Secret called `inference-s3-credentials`. At startup the service downloads the weights, compares their SHA-256 with `model.sha256`, and only then loads them and starts reporting ready.

A small model will sometimes produce broken output, so the service does not trust it. The text is read with a balanced-brace parser and checked against the schema. If it cannot be parsed or has the wrong keys, the endpoint returns HTTP 500 with `{"error": "invalid_model_output"}` instead of passing bad data on. To test this path I added a switch: when `FAULT_INJECT=1`, the service replaces the model output with the

string `not json` before validation, which must give the 500. The unit tests in `tests/test_service.py` check this.

For the image (`service/Dockerfile`) I started from `python:3.11-slim` and installed the CPU-only PyTorch wheels from `https://download.pytorch.org/whl/cpu`, with every dependency pinned. The default PyTorch install pulls in large CUDA packages that are useless on a CPU-only cluster and would have made the image far bigger. Keeping the weights out of the image brought it to 1.87 GB, under the 2.0 GB limit. Version pinning was needed for a different reason: the PyTorch and Transformers versions I tried first did not work together, and fixing exact versions (listed in Section 3) solved it and also keeps the Docker build repeatable.

To check that the service fits its limits, I sent 50 sequential requests to a container limited to 1 CPU and 1 GiB of memory. All 50 returned HTTP 200 with no failures or out-of-memory kills. Latency was 0.530 s minimum, 0.742 s average and 0.885 s maximum. `kubectl top pods -n app` showed 124m CPU and 493 MiB on one pod and 2m CPU and 466 MiB on the other. The peak of 493 MiB is 48.1% of the memory limit (measured).

5. Task 4: Kubernetes Deployment, Scaling and Reliability

In the `app` namespace I created a Deployment with two replicas of `command-parser:v1` (CPU request 250m and limit 1000m, memory request 512Mi and limit 1Gi) with startup, liveness (`/healthz`) and readiness (`/readyz`) probes. A ClusterIP Service exposes port 8000, a Traefik Ingress sends path `/` to it, and a HorizontalPodAutoscaler is set to a minimum of 1 and a maximum of 2 replicas at 70% CPU. The CPU request is the baseline the HPA uses: the busy pod used 124m against a 250m request, which is about 50%, so the HPA would not be expected to react at this load. I did not run a test that triggered scaling, so I treat the HPA as configured but not demonstrated.

Rolling update and rollback. A model takes around ten seconds to load, so a rolling update can easily send traffic to a pod that is not ready. This is the reason `/readyz` returns 503 until the weights are loaded: Kubernetes only routes to a new pod after that point. While load was running, I updated the Deployment to model v2 with `kubectl set env deployment/command-parser MODEL_VERSION=v2 MODEL_URI=s3://models/v2/`. The rollout completed without dropped requests, and `kubectl rollout undo` brought back `X-Model-Version: v1`, again with 0.0% failed requests (measured).

Self-healing. With traffic running I deleted a pod using `kubectl delete pod`. Kubernetes started a replacement, which downloaded the weights from SeaweedFS, passed the readiness probe and brought the replica count back to 2. All 33 requests in the test succeeded, i.e. 0.0% dropped (measured).

Cold start. I deleted all pods in the `app` namespace together, three times, and timed how long they took to become Ready: 10.72 s, 10.73 s and 10.76 s, a median of 10.73 s (measured). The logs show each pod downloading the model files, printing that the checksum was verified, loading the weights and then turning ready.

Load balancing. I ran Locust through the Traefik Ingress with both replicas Ready. The two pods served 146 and 114 requests, which is 56.15% and 43.85% of those 260 requests, so both are well above the required 30% minimum. (The Locust run itself reported 302 requests, see Section 8; the pod-level counts add up to 260, so I could not account for the difference from my records.)

One limit of this setup is that the whole cluster lives on one laptop. The k3d nodes are Docker containers that share the same CPU, memory and disk, so I could not show real node failure; if the host went down, everything would go down with it.

6. Task 5: Subnetting and Network Access Control

The first part was to split the address block `10.20.0.0/16` into four subnets using VLSM. The method I followed is to sort the requirements from largest to smallest, find the smallest prefix that fits using usable hosts

$= 2^{32-\text{prefix}} - 2$, and place each block right after the previous one so nothing overlaps. For the app subnet, $2^{10} - 2 = 1{,}022 \geq 1{,}000$, so the prefix is /22. The same steps give /24, /25 and /27 for the others.

Table 3. VLSM allocation inside 10.20.0.0/16.

Subnet	Network	Prefix	Usable range	Broadcast	Usable hosts
app (1,000 needed)	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

The second part was a Network ACL for the public subnet, where the load balancer sits, written in `network/nacl.json`. Table 4 lists the rules. Rules are checked in ascending order and the first match wins.

Table 4. NACL rules for the public subnet.

Direction	Rule	Action	Protocol / port	Source or destination
Inbound	90	DENY	All	198.51.100.0/24
Inbound	100	ALLOW	TCP 80	0.0.0.0/0
Inbound	110	ALLOW	TCP 443	0.0.0.0/0
Inbound	120	ALLOW	TCP 22	10.20.5.128/27 (management)
Inbound	130	ALLOW	TCP 1024–65535	10.20.0.0/22 (app)
Inbound	140	DENY	All	Everything else
Outbound	100	ALLOW	TCP 8000	10.20.0.0/22 (app)
Outbound	110	ALLOW	TCP 1024–65535	0.0.0.0/0
Outbound	120	DENY	All	Everything else

Two points are worth remembering for a viva. First, a NACL is stateless. A client connects to port 80, and the reply goes back to the client's temporary (ephemeral) port in the range 1024–65535. The NACL does not remember the connection, so the reply is dropped unless an outbound rule such as 110 allows that range. A stateful firewall would allow it automatically. Second, order matters: the deny rule for 198.51.100.0/24 is number 90. If it were number 150, a packet from 198.51.100.9 would match rule 100 (`0.0.0.0/0` on port 80) first and be allowed. I tested the rules with `tools/nacl_sim.py` on `network/packets_sample.jsonl`, and all 6 of 6 sample packets got the correct decision (100%, measured). This is a simulator check on a small sample, not a test on a real cloud network.

7. Task 6: Reproducibility, Testing and CI/CD

I wanted the whole project to be rebuildable from the repository with a few commands, so the `Makefile` has these targets: `make up` creates the k3d cluster, deploys SeaweedFS in `data`, and deploys the service, ingress and HPA in `app`; `make down` deletes the cluster; `make train SUBSET=50` runs a short training to check the code; and `make test` runs the tests in `tests/test_service.py`. There are five unit tests, covering `/healthz`, `/readyz`, `FAULT_INJECT=1`, the response headers and schema validation.

The GitHub Actions workflow (`.github/workflows/ci.yml`) has five stages: (1) linting with flake8 and ruff; (2) the unit tests; (3) model evaluation, which generates predictions on `data/validation.jsonl`; (4) a quality gate that runs `tools/score.py predictions.jsonl data/validation.jsonl --min-validity 95` and fails the pipeline if JSON validity is below 95%; and (5) a gitleaks scan over the Git history to make sure no credentials were committed.

To show that the quality gate really blocks a bad model, I ran the pipeline once with the untrained Pythia-70M. It produced 0% valid JSON, the gate exited with status 1 and the pipeline failed. The run with the trained student (99% validity) passed. Both runs are on GitHub:

- Failed run (untrained model): https://github.com/NirajS2118/cloud-assignment/actions/runs/1173928101
- Passing run (trained student): https://github.com/NirajS2118/cloud-assignment/actions/runs/1173934208

8. Task 7: Performance, Cost Estimation and Project Review

Measured performance. I used Locust with 4 concurrent users through the Traefik Ingress. The median latency (p50) was 740 ms and the 95th percentile (p95) was 900 ms, well under the 5,000 ms target. Throughput was 5.13 requests per second and the error rate was 0.0%. Pod usage was 124m CPU / 493 MiB and 2m CPU / 466 MiB, and the median cold start was 10.73 s. Under concurrent load, requests are generated on CPU with a 1-core limit, and each takes about 0.74 s, so extra concurrent users mostly wait in a queue behind each other. The reported run used 4 users.

Estimated cost. This part is an estimate, not a measurement. With the rate card in `data/rate_card.json` (₹3.50 per vCPU-hour and ₹0.45 per GiB-hour) and a limit of 1 vCPU and 1 GiB per replica:

$$\text{Cost per replica} = 1 \times 3.50 + 1 \times 0.45 = ₹3.95 \text{ per hour}$$

$$\text{Two replicas} = 2 \times 3.95 = ₹7.90 \text{ per hour}$$

$$\text{Requests per hour} = 5.13 \times 3{,}600 = 18{,}468$$

$$\text{Cost per 1,000 requests} = (7.90 / 18{,}468) \times 1{,}000 = ₹0.4278 \approx ₹0.43$$

This assumes both replicas run all the time and that the service keeps handling 5.13 requests per second. The cost per 1,000 requests would go up at lower traffic, because the hourly cost does not change.

SLO and error budget. My SLO is that at least 99.0% of `/parse` requests return HTTP 200 within 5,000 ms. For one Locust run of 302 requests, the allowed failures are $302 \times 0.01 = 3.02$, so about 3 requests. I observed no infrastructure-level failures.

Table 5 reviews each area, with the evidence I measured, the limit I found, and an improvement I would try. The last column is only proposed; I have not implemented or tested any of it.

Table 5. Project review (evidence is measured; improvements are proposed only).

Area	Evidence	Limitation	Proposed improvement
Data and storage	2,435 of 3,000 kept; 359 rejected for non-substring slots	Teacher sometimes rephrased slot values instead of copying text	Grammar-constrained JSON decoding for the teacher (e.g. Outlines or Guidance)
Model quality	99% JSON validity; 16% full-frame accuracy	70M parameters struggle with date and intent	Fine-tune a ~1B model such as Llama-3.2-1B-Instruct with LoRA
Latency and throughput	740 ms p50, 900 ms p95, 5.13 req/s at 4 users	FP32 decoding on CPU limits throughput	Export to ONNX Runtime with INT8 quantization
Resources	493 MiB peak of 1 GiB; 124m CPU under load	PyTorch runtime uses much memory even when idle	A lighter engine such as llama.cpp / GGML
Reliability	0.0% failures on pod deletion; 10.73 s cold start	Each pod downloads ~281 MB of weights at start	Shared volume cache or weights in a warm image
Security and cost	6/6 NACL packets correct; ₹0.43 per 1,000 requests (est.)	Fixed replicas cost ₹7.90 per hour even when idle	KEDA or Knative for scale-to-zero

9. Limitations and Future Improvements

Beyond Table 5, there are a few limits that affect how far the results can be trusted. The validation set has only 100 commands, so small differences between models are not reliable. The student learned from the teacher's labels, and the teacher itself reached only 45% full-frame accuracy, so the student's ceiling is limited by the teacher. The cluster runs on one machine, so reliability tests show Kubernetes behaviour but not real hardware failure. The HPA is configured but I have no run where it scaled, and the cost estimate is based on a short load test with 4 users. If I continued, I would first improve the data (constrained teacher decoding and a larger validation set), then try a slightly larger student, and only after that spend time on speed (quantization) and cost (scale-to-zero).

Table 6. Problems met during the project and what was done (see the sections named for details).

Problem	What I did
PyTorch and Transformers versions did not work together (Section 4)	Pinned exact versions (PyTorch 2.5.1, Transformers 4.46.3) in the training config and the Dockerfile
Default PyTorch install brings unneeded CUDA packages (Section 4)	Installed CPU-only wheels; weights kept out of the image; final image 1.87 GB

Problem	What I did
Small Pythia-70M repeats text; untrained model gave 0% valid JSON (Section 3)	Trained on teacher JSON (99% validity after training); output limited by `MAX_NEW_TOKENS=100`; JSON picked out with a balanced-brace parser
Invalid JSON or wrong schema from a model (Sections 2 and 4)	Teacher records filtered by rules (a) to (d); service validates against `schema.json` and returns HTTP 500 on bad output
Rolling updates could send traffic to a pod still loading (Section 5)	`/readyz` returns 503 until the weights are loaded; update and rollback tests showed 0.0% failures
Requests queue under concurrent load (Section 8)	Load test run with 4 users; p95 900 ms against a 5,000 ms target; HPA allows up to 2 replicas (scaling not demonstrated)

10. Conclusion and Learning Outcomes

The project produced a working pipeline from raw commands to a deployed service. The distilled Pythia-70M student gives valid JSON 99% of the time, but only 16% of its full outputs are completely correct, so it is good at the format and weaker at the meaning. The service ran within its 1 GiB limit (493 MiB peak), recovered from a deleted pod and from a rolling update with no dropped requests in the tests I ran, and met the latency target (p95 900 ms against 5,000 ms).

The main things I learned were these. Cleaning and accounting for every record matters, because 18.8% of the teacher's output could not be used. Distillation method is decided by what the teacher exposes, and here only text was available. Liveness and readiness probes do different jobs, and readiness is what protects users during updates. A stateless NACL needs rules for return traffic and its rule order decides the result. Finally, a CI quality gate is only convincing once you have seen it fail on a bad model.